

Plano de Melhoria Estrutural: Arquitetura de Diretórios e Otimização de Contexto

Estratégia para eliminação de duplicidades, centralização de skills via symlinks e otimização de orquestração agêntica

Auditoria técnica do pipeline

2026-08-05

Plano de Melhoria Estrutural: Arquitetura de Diretórios e Otimização de Contexto

Este plano propõe uma reestruturação profunda e de baixo impacto na arquitetura de arquivos e diretórios da Fábrica de Materiais de Comunicação. O objetivo principal é otimizar o consumo de tokens nas inicializações dos agentes, eliminar a duplicidade física de skills e unificar a governança do ecossistema multiplataforma (Claude, Gemini, Cursor, VSCode, CodeBuddy e Qoder).

Diagnóstico do Estado Atual e Desperdício de Contexto

O ecossistema da fábrica agêntica atualmente enfrenta dois gargalos estruturais de redundância:

1. **Duplicidade em Arquivos de Orquestração (CLAUDE.md, GEMINI.md, AGENTS.md):**
 - Os três arquivos principais de governança compartilham exatamente as mesmas ~150 linhas iniciais de cabeçalho, regras invioláveis e diagramas de arquitetura.
 - **Impacto:** Sempre que um agente (Claude ou Gemini) inicializa ou é acionado, ele consome essas ~150 linhas redundantes. Em sessões longas, isso se traduz em milhares de tokens desperdiçados desnecessariamente, aumento de latência e elevação do custo financeiro operacional por chamada.
 2. **Duplicidade Física de Skills entre Agentes:**
 - As mesmas pastas de skills (como debug-issue, explore-codebase, refactor-safely e review-changes) estão copiadas fisicamente em múltiplos diretórios raiz de plataformas distintas: .claude/skills/, .gemini/skills/ e .codebuddy/skills/.
 - **Impacto:** Riscos graves de dessincronização operacional. Se uma melhoria é aplicada em uma skill de depuração sob o contexto do Claude, ela não se reflete automaticamente no Gemini ou no CodeBuddy, gerando manutenção manual redundante e bugs silenciosos por defasagem de versão.
-

Proposta de Solução: Divisão de Responsabilidades de Orquestração

Para eliminar o desperdício de tokens, propõe-se a remoção de todo o conteúdo duplicado dos arquivos de governança, especializando cada um de acordo com seu público-alvo e plataforma. Toda a base técnica comum será consolidada no arquivo de arquitetura central.

Tabela de Especialização de Arquivos de Orquestração

Arquivo	Papel Estratégico	Conteúdo Exclusivo	Tamanho Estimado
AGENTS.md	Bíblia Conceitual da Fábrica (Única fonte de verdade arquitetural)	Visão geral da fábrica agêntica, regras invioláveis de negócio, fluxo do pipeline de ponta a ponta, tabela de materiais e módulos, pré-requisitos de ambiente e catálogo de skills.	~180 linhas
CLAUDE.md	Orquestração Es- trita do Claude (Focado no Claude Code e Claude CLI)	Comandos exclusivos do Claude (/esbocar, /gerar-*), instruções de uso das ferramentas nativas do Claude, diretivas de auto-correção específicas e links de referência para AGENTS.md.	~30 linhas
GEMINI.md	Orquestração Es- trita do Gemini (Focado no Gemini CLI e AI Studio)	Padrões de economia de tokens do Gemini (Caveman Thinking), regras de pre-flight do Gemini, hooks de sessão (crg-session-start.sh), uso do code-review-graph e links para AGENTS.md.	~35 linhas

Nota: Ao adotar essa especialização, reduzimos o overhead de inicialização do Claude Code de ~210 linhas para apenas ~30 linhas, gerando uma economia de mais de 80% de tokens de preâmbulo em cada interação.

Arquitetura de Diretórios e Técnicas de Symlinks

A proliferação de diretórios de configuração de agentes na raiz (.claude/, .gemini/, .codebuddy/, .qoder/, .cursor/, .kiro/, .vscode/) pode ser organizada sem perder a compatibilidade com cada plataforma.

Centralização de Skills na Raiz do Workspace

Propõe-se a criação de um diretório centralizado skills/ diretamente na raiz do projeto (ou .skills/ para manter oculto se preferido). Todas as skills comuns do ecossistema serão movidas e mantidas exclusivamente neste local.

```
C:\Users\trcnologia\Desktop\proj_fabrica-comunicacao\
├── skills/                               ← ÚNICA FONTE DE VERDADE DE SKILLS COMUNS
│   ├── debug-issue/
│   ├── explore-codebase/
│   ├── refactor-safely/
│   └── review-changes/
├── .claude/
│   └── skills/                          ← Link Simbólico (Symlink) para ../../skills/
├── .gemini/
│   └── skills/                          ← Link Simbólico (Symlink) para ../../skills/
├── .codebuddy/
│   └── skills/                          ← Link Simbólico (Symlink) para ../../skills/
```

Técnicas de “@” e Mapeamento Dinâmico de Workspace

1. Evitar Caminhos Absolutos de Máquina:

- Atualmente, os arquivos de configuração (como os caminhos de skills listados no sistema) contêm referências absolutas à máquina do operador atual (ex.: C:\Users\trcnologia\...). Isso quebra a portabilidade da fábrica para outros desenvolvedores ou servidores CI/CD.
- **Solução:** Substituir todas as referências rígidas por variáveis de ambiente ou caminhos dinâmicos baseados na raiz do workspace.

2. Mapeamento de Aliases (“@”):

- Utilizar a técnica de mapeamento lógico para identificar caminhos internos. Em ferramentas de IDE e prompts de orquestração, estabelecer convenções curtas:
 - @workspace → Raiz do projeto.
 - @skills → Pasta central skills/ da raiz.
 - @brand → Pasta de identidade visual brand/.
 - @output → Pasta de compilação output/.

Outras Sugestões de Organização e Governança

1. Script de Setup Automático (scripts/setup-workspace.py)

Para viabilizar o uso de symlinks em ambientes híbridos (Windows e Unix/macOS) sem exigir esforço manual do operador, propõe-se um script em Python que configura dinamicamente o workspace.

Funcionalidades do script: - Detecta o Sistema Operacional e os privilégios do terminal. - Se no Windows, utiliza chamadas mklink ou APIs nativas do Python os.symlink (requer Modo Desenvolvedor ativo ou privilégios de Administrador). - Cria automaticamente os symlinks de skills/ centrais para as pastas .claude/skills/, .gemini/skills/, .codebuddy/skills/ e .qoder/skills/. - Limpa pastas físicas redundantes remanescentes que foram substituídas pelos links lógicos.

2. Modularização de Scripts de Apoio

A pasta scripts/ atualmente mistura utilitários de compilação, scripts de validação de design e auxiliares de empacotamento. Sugere-se uma separação organizacional interna na pasta de scripts:

- scripts/compiladores/ → compilar-pdf.py, compilar-html.py, compilar-arte.py, pdf_typst.py.
- scripts/validadores/ → validar-html.py, validar-pdf.py, validar-dimensoes.py, validar-logo.py, validar-transparencia.py, validar-textos.py, validar-design-tokens.py.
- scripts/pipeline/ → pool-materiais.py, auditar-projeto.py, empacotar-projeto.py, verificar-consistencia-pipeline.py, preflight-compatibilidade-slug.py.

Nota: Para garantir que nenhuma importação relativa ou chamada externa de ferramentas quebre, o script de bootstrap criará os symlinks necessários ou o projeto manterá caminhos de inclusão mapeados no sys.path de scripts/parametros_projeto.py.

3. Consolidação de Arquivos de Configuração MCP

Existem múltiplos arquivos .mcp.json espalhados pelo projeto: .mcp.json na raiz, .cursor/mcp.json, .vscode/mcp.json, .qoder/mcp.json, .kiro/settings/mcp.json. - **Melhoria:** Centralizar as definições de MCP servers lógicos em um único arquivo de referência no repositório (docs/mcp-configuration-template.json) e fazer com que o script de setup sincronize ou copie esse arquivo para as pastas ocultas específicas de cada ferramenta parceira.

Plano de Execução e Transição

A migração para o novo modelo de diretórios e orquestração pode ser realizada de forma segura e incremental em 3 etapas consecutivas, garantindo retrocompatibilidade total com as compilações em andamento.

Cronograma de Ação

- | | |
|---|---|
| Etapas 1 (Imediata)
(Economia de tokens) | → Especialização de CLAUDE.md, GEMINI.md e AGENTS.md |
| Etapas 2 (Curto Prazo)
de Bootstrap Symlinks | → Centralização de Skills na Raiz e Criação do Script |
| Etapas 3 (Médio Prazo) | → Modularização da pasta scripts/ e Unificação de configurações MCP |

Etapas 1: Refatoração da Camada de Orquestração (Tokens)

- Extrair o cabeçalho comum atual para AGENTS.md.

- Limpar `CLAUDE.md`, reduzindo seu tamanho operacional ao mínimo específico para o Claude.
- Limpar `GEMINI.md`, mantendo as instruções específicas do Gemini e as regras de economia Caveman.
- Validar se os agentes continuam operando normalmente sem perda de contexto funcional.

Etapa 2: Implementação da Arquitetura Lógica de Skills (Symlinks)

- Criar a pasta central `skills/` na raiz.
- Mover as skills comuns para a pasta central.
- Desenvolver o script de bootstrap `scripts/setup-symlinks.py`.
- Rodar o script para gerar os symlinks nas pastas de agentes específicas.
- Testar e validar a execução das ferramentas do ecossistema sob a nova estrutura lógica.

Etapa 3: Governança de Configurações e Arquivos MCP

- Centralizar os templates MCP e unificar parâmetros dinâmicos.
- Aplicar a modularização opcional de subpastas em `scripts/` se a complexidade do ecossistema crescer.
- Adicionar rotina de limpeza de caches de compilação temporários em `output/` para manter o repositório leve.